ALPE FORMACIÓN · SANTANDER

Edición 2026 · PHP 8.x

CURSO DE PHP · CLASE 01

El mundo de la programación hoy



Ramón Abramo

+25 años de experiencia docente · Santander, Cantabria

ALPE

FORMACIÓN PROFESIONAL

🧭 Contenido de la clase

Buenas, soy **Ramón Abramo**, profesor con más de 25 años de experiencia docente, y hoy empezamos este viaje juntos. 👋

Esta primera clase es **solo teoría**. No vamos a instalar nada ni a escribir ni una línea de código. Mi objetivo es que, al terminar, sepáis situaros: qué tipos de programación existen, qué hace cada tecnología y qué camino vamos a recorrer durante el curso. Cuando lleguemos a PHP, a MySQL o a Laravel, ya sabréis dónde encaja cada pieza.

Así que tranquilos. Hoy toca escuchar, preguntar y hacerse una idea del mapa.

- 🤔 ¿Qué es programar?



- 📘 Programación de escritorio y programación web



- 🔍 Cómo funciona la web: cliente y servidor



- 🚀 La analogía del restaurante: cómo trabaja PHP



- 👌 Frontend, backend y fullstack



- 🚀 Las tecnologías que vamos a usar



- 👍 Lo que os vais a encontrar en el curso



- 😮 Vocabulario de hoy





CLASE 01

🤔 ¿Qué es programar?

Programar es **darle instrucciones a un ordenador** para que haga una tarea. Nada más... y nada menos.

El ordenador es muy rápido, pero no piensa: hace exactamente lo que le decimos, en el orden en que se lo decimos. Si la instrucción está mal, el resultado estará mal. Por eso os repetiré muchas veces que el ordenador nunca se equivoca: nos equivocamos nosotros al explicarle las cosas. 😉

Esas instrucciones se escriben en un **lenguaje de programación**: PHP, JavaScript, Python, Java, C#... Cada uno tiene sus reglas (su *sintaxis*) y está pensado para unos usos concretos.



CLASE 01

📘 Programación de escritorio y programación web

Programación web vs. programación de escritorio

Dos formas de llevar un programa al usuario

🌐

APLICACIÓN WEB

Se usa desde el navegador

- No se instala: basta con una URL
- El programa vive en un **servidor**
- Funciona en Windows, Mac, Linux y móvil
- Se actualiza una vez y llega a todos
- Ejemplos: Gmail, Amazon, Moodle, Netflix

🖥️

APLICACIÓN DE ESCRITORIO

Se instala en el ordenador

- Hay que descargarla e instalarla
- Se ejecuta en **tu equipo**
- Suele depender del sistema operativo
- Cada usuario tiene que actualizarla
- Ejemplos: Word, Photoshop, VS Code, juegos

🎯 **En este curso nos centramos en la programación web**: aplicaciones que se ejecutan en un servidor y que cualquiera usa desde su navegador.

Realizado por Ramón Abramo

Hoy en día hay dos grandes formas de hacer llegar un programa al usuario.

🖥️ Aplicaciones de escritorio

Son las que **descargáis e instaláis** en vuestro ordenador: Word, Photoshop, un juego, el propio editor de código que usaremos. Se ejecutan en vuestro equipo y suelen depender del sistema operativo (una versión para Windows, otra para Mac...). Si sale una actualización, cada usuario tiene que instalarla.

🌐 Aplicaciones web

Son las que usáis **desde el navegador**: Gmail, Amazon, Netflix, la plataforma de vuestro banco o Moodle. No instaláis nada: escribís una dirección y listo. El programa vive en un **servidor** (un ordenador que está en algún lugar de internet, encendido siempre) y vuestro navegador solo muestra el resultado.

Fijaos en la ventaja: funciona igual en Windows, en Mac, en Linux o en el móvil, y cuando el programador la actualiza, la actualiza para todos a la vez.

	Escritorio 🖥️	Web 🌐
¿Hay que instalarla?	Sí	No, basta con el navegador
¿Dónde se ejecuta?	En vuestro ordenador	En un servidor
¿Depende del sistema operativo?	Normalmente sí	No
Actualizaciones	Cada usuario la actualiza	Se actualiza una vez para todos
Ejemplos	Word, Photoshop, juegos	Gmail, Amazon, Moodle



🎯 **En este curso nos centramos en la programación web.** Vamos a construir aplicaciones que se ejecutan en un servidor y que cualquiera puede usar desde su navegador.



CLASE 01

🔍 Cómo funciona la web: cliente y servidor



Cada vez que abrís una página web ocurre siempre lo mismo, aunque no lo veáis:

1. 🧑‍💻 **El cliente pide.** Vuestro navegador (Chrome, Firefox, Edge...) envía una **petición** al servidor: «dame la página `index.php`».
2. 🗄️ **El servidor trabaja.** Un programa llamado **servidor web** (por ejemplo, Apache) recibe la petición. Si la página es de PHP, PHP ejecuta el código y, si hace falta, consulta la base de datos MySQL.
3. 📄 **El servidor responde.** Devuelve al navegador una **respuesta** con la página ya terminada, en HTML.
4. 👀 **El navegador muestra.** Interpreta ese HTML, le aplica el diseño (CSS) y la interactividad (JavaScript), y os enseña la página.

El idioma en el que hablan navegador y servidor se llama **HTTP** (o HTTPS, su versión segura, la del candado 🔒) [5].

Ahora, un detalle que os va a ahorrar muchos dolores de cabeza: **el navegador nunca ve el código PHP**. Solo recibe el HTML que PHP ha generado. Lo que pasa en el servidor, se queda en el servidor.



CLASE 01

🚀 La analogía del restaurante: cómo trabaja PHP

Lo de «petición» y «respuesta» suena muy técnico, así que os lo cuento como lo cuento siempre en clase: **imaginad que entráis a cenar a un restaurante** bien organizado. Intervienen cuatro personajes que se coordinan a la perfección.



	En el restaurante	En la web	Qué hace
🧑	**El comensal** en la mesa	El **navegador** (el cliente)	Abre la carta (escribe la dirección) y pide lo que quiere. **Nunca entra en la cocina**: solo se come el plato cuando se lo sirven terminado
🤵	**El camarero**	El protocolo **HTTP**	Lleva la comanda de la mesa a la cocina (la **petición**) y vuelve con el plato (la **respuesta**)
👨‍🍳	**El chef** en la cocina	El **servidor web** (Apache) y el intérprete de **PHP**	Lee la receta (vuestro código PHP), coge los ingredientes, cocina y emplata el resultado en **HTML**
🗄️	**La despensa** con candado	La base de datos **MySQL**	Guarda los ingredientes (usuarios, productos, notas…). **Solo el chef tiene la llave**

Una cena, paso a paso

Pongamos que en la web de una academia pedís la lista de cursos, `cursos.php`:

1. 🧑 **Pedís.** Escribís la dirección en el navegador: el comensal pide «la lista de cursos».
2. 🤵 **El camarero lleva la comanda.** El navegador envía una **petición HTTP** al servidor.

3. 👨‍🍳 **El chef lee la receta.** Apache ve que es un archivo `.php` y se lo pasa a PHP, que ejecuta el código línea a línea.
4. 🗄️ **El chef va a la despensa.** El código le pide a MySQL los cursos que hay guardados.
5. 🍽️ **El chef emplata.** PHP mezcla esos datos con el diseño y prepara una página **HTML** terminada.
6. 🤵 **El camarero sirve el plato.** El servidor envía la **respuesta HTTP** con ese HTML.
7. 😋 **Coméis.** El navegador pinta la página, con su CSS y su JavaScript.

Las tres ideas que os tenéis que llevar

- **El comensal nunca ve la receta.** El navegador recibe el plato cocinado (HTML), nunca el código PHP. Por eso, si en el navegador hacéis clic derecho y «Ver código fuente», no veréis ni una línea de PHP.
- **Sin cocina no hay plato.** Si hacéis doble clic en un archivo `.php` desde el explorador de Windows, no funciona: lo estáis abriendo sin chef. Los archivos PHP **tienen que pasar por un servidor**. Para eso instalaremos Laragon en la clase 2: una cocina completa dentro de vuestro ordenador.
- **La despensa tiene candado.** El usuario no puede tocar la base de datos directamente; siempre pasa por el código PHP, que decide qué se puede sacar y qué no. Gran parte de la **seguridad** de una web consiste en que ese chef no se deje engañar.



💡 Vamos a volver a este restaurante muchas veces durante el curso: el camarero cuando veamos los formularios y los códigos de estado HTTP, la despensa cuando lleguemos a las bases de datos… Si en algún momento os perdéis, pensad: **¿quién hace esto en el restaurante?**



CLASE 01

👌 Frontend, backend y fullstack

Frontend, backend y fullstack

Las dos caras de una aplicación web... y quien domina las dos

🎨

FRONTEND · LO QUE SE VE

En el navegador del usuario

- Diseño, colores, botones, menús
- Que la página sea cómoda y se adapte al móvil
- Tecnologías: **HTML, CSS y JavaScript**

⇄

⚙️

BACKEND · LO QUE NO SE VE

En el servidor

- Lógica: usuarios, pedidos, cálculos
- Guardar y leer datos de la base de datos
- Tecnologías: **PHP** (y Laravel) + **MySQL**



FULLSTACK

Desarrollador que trabaja en las **dos partes**: sabe montar la interfaz y también programar el servidor y la base de datos. Ese es el perfil al que vamos a llegar en el curso.

🍽️ **Pensad en un restaurante:** el frontend es la sala (mesas, carta, camarero) y el backend es la cocina (donde se prepara todo y está la despensa, que sería la base de datos).

Realizado por Ramón Abramo

Una aplicación web tiene dos caras, y en el mundo laboral cada una tiene su nombre.

🎨 Frontend: lo que se ve

Es todo lo que ocurre **en el navegador del usuario**: el diseño, los colores, los botones, los menús, que la página se adapte bien al móvil. El frontend trabaja con **HTML, CSS y JavaScript**.

⚙️ Backend: lo que no se ve

Es lo que ocurre **en el servidor**: comprobar el usuario y la contraseña, calcular el total de un pedido, guardar un registro, enviar un correo. El backend trabaja con lenguajes como **PHP** y con bases de datos como **MySQL**.

🦸 Fullstack: las dos cosas

Un desarrollador **fullstack** sabe moverse en las dos partes: monta la interfaz y también programa el servidor y la base de datos. No hace falta ser un experto absoluto en todo, pero sí entender cómo encajan las piezas.



🍽️ **Volvamos al restaurante.** El frontend es la sala: las mesas, la carta, cómo os presentan el plato. El backend es la cocina, donde se prepara todo y donde está la despensa (la base de datos). El cliente nunca entra en la cocina, pero sin cocina no hay comida.

Nuestro curso está centrado en el **backend con PHP**, aunque usaremos HTML, CSS y algo de JavaScript para que nuestras páginas tengan buena cara. Al final, con Laravel, tendréis una visión bastante completa de cómo se construye una aplicación de principio a fin.





Para entender qué hace cada una, os propongo una analogía: construir una casa. 🏠

🧱 HTML: la estructura

HTML (*HyperText Markup Language*) no es un lenguaje de programación, es un **lenguaje de marcado**. Sirve para decir *qué hay* en la página: un título, un párrafo, una imagen, un enlace, un formulario. Son los **ladrillos y las paredes** de la casa [2].

🎨 CSS: el diseño

CSS (*Cascading Style Sheets*) dice *cómo se ve* cada cosa: colores, tamaños, tipos de letra, márgenes, posición. Es la **pintura y la decoración** [3]. El mismo HTML con otro CSS puede parecer una web totalmente distinta.

⚡ JavaScript: la interactividad

JavaScript sí es un lenguaje de programación, y se ejecuta **en el navegador**. Hace que la página reaccione: un menú que se despliega, un aviso si dejáis un campo vacío, una galería de fotos que se mueve. Es la **instalación eléctrica**: interruptores, luces, timbre [4].

Ojo con el nombre. JavaScript **no tiene nada que ver con Java**, por mucho que se parezcan. Es como el coche y el cochinillo. 🐷

🐘 PHP: la lógica del servidor

PHP es el lenguaje que vamos a aprender en profundidad. Se ejecuta **en el servidor** y su trabajo es generar páginas dinámicas: páginas cuyo contenido cambia según el usuario, la hora o los datos guardados [1]. Lo usan WordPress, Moodle y frameworks como Laravel, así que una parte enorme de la web funciona con PHP. Es el **cerebro** de la casa: decide qué hay que hacer en cada momento.

🐬 MySQL: los datos

MySQL es un **sistema gestor de bases de datos**. Guarda la información de forma ordenada, en tablas: usuarios, productos, pedidos, notas... Para hablar con MySQL se usa otro lenguaje, **SQL** [6]. Es el **archivador** de la casa: todo lo que hay que recordar se guarda ahí.

🚀 Laravel: el framework

Un **framework** es un conjunto de herramientas y una forma organizada de trabajar que nos ahorra reinventar la rueda. **Laravel** es el framework de PHP más utilizado hoy en día, y será la meta final del curso [7]. Pero no os adelantéis: primero hay que aprender a andar con PHP «a pelo».

📋 Resumen en una tabla

Tecnología	¿Qué hace?	¿Dónde se ejecuta?	¿Es lenguaje de programación?
🧱 HTML	Estructura y contenido	Navegador	No (lenguaje de marcado)
🎨 CSS	Diseño y presentación	Navegador	No (lenguaje de estilos)
⚡ JavaScript	Interactividad	Navegador	Sí
🐘 PHP	Lógica del servidor	Servidor	Sí
🐬 MySQL	Guardar y consultar datos	Servidor	Es un gestor de bases de datos; se maneja con SQL



CLASE 01

👍 Lo que os vais a encontrar en el curso

Os cuento el camino para que sepáis hacia dónde vamos:

1. 🛠️ **Preparar el entorno.** Instalaremos **Laragon**, que nos da en un solo paquete un servidor web (Apache), PHP y MySQL en nuestro propio ordenador. Así podremos programar como si tuviéramos un servidor de verdad.
2. 🧩 **Preparar el editor.** Configuraremos **Visual Studio Code** con las extensiones que nos harán la vida más fácil.
3. 🐘 **PHP desde cero.** Variables, condiciones, bucles, arrays, funciones y formularios. La base de todo.
4. 🐬 **Bases de datos con MySQL.** Aprenderemos SQL y conectaremos PHP con MySQL.
5. 🧱 **Programación orientada a objetos.** Clases, objetos, herencia... La forma en que se organiza el código profesional, y el corazón de este curso.
6. 🏗️ **Nuestro propio MVC.** Construiremos un pequeño framework para entender qué hay «debajo del capó».
7. 🚀 **Laravel.** Con todo lo anterior, aprenderemos a desarrollar aplicaciones completas con el framework más usado de PHP.

Iremos paso a paso. Cada clase se apoya en la anterior, así que si algo no os queda claro, preguntad. No existen las preguntas tontas: tontería es quedarse con la duda.



CLASE 01

😮 Vocabulario de hoy

Término	Significado
Cliente	El programa que pide la página; normalmente, el navegador
Servidor	Ordenador (y programa) que recibe peticiones y devuelve respuestas
Servidor web	Programa que atiende las peticiones HTTP; en nuestro caso, Apache
HTTP / HTTPS	Protocolo con el que se comunican navegador y servidor; la S indica conexión cifrada
Frontend	Parte de la aplicación que se ve y se ejecuta en el navegador
Backend	Parte de la aplicación que se ejecuta en el servidor
Fullstack	Perfil que trabaja tanto en frontend como en backend
Base de datos	Lugar donde se guarda la información de forma organizada
SQL	Lenguaje para consultar y modificar bases de datos
Framework	Conjunto de herramientas y normas para desarrollar más rápido y de forma ordenada

Próxima clase: instalamos el entorno de desarrollo con Laragon. 👉 Clase 2 · Entorno de desarrollo con Laragon

📚 Fuentes y documentación oficial

1. PHP: ¿Qué es PHP? — Manual oficial de PHP
2. MDN Web Docs: HTML
3. MDN Web Docs: CSS
4. MDN Web Docs: JavaScript
5. MDN Web Docs: Generalidades del protocolo HTTP
6. MySQL 8.0 Reference Manual: What is MySQL?
7. Laravel: documentación oficial